

Project Files & Folders

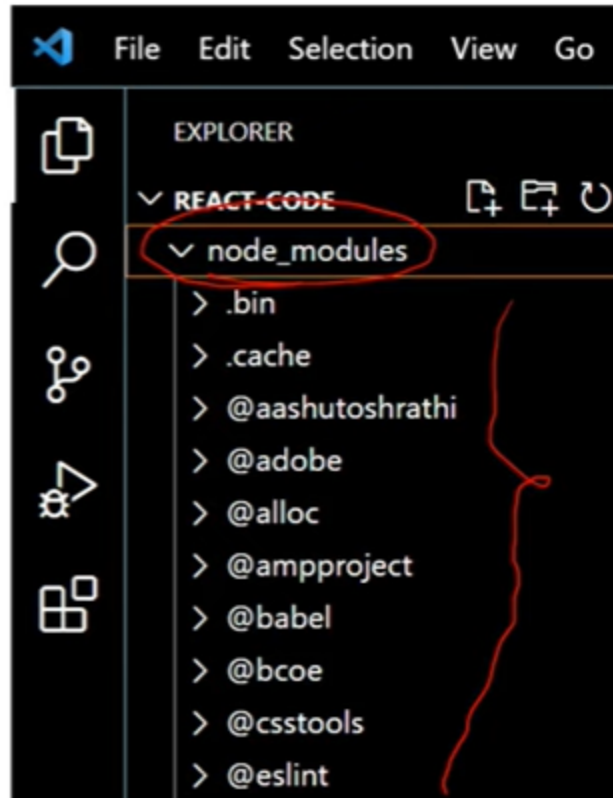
▼ Q1. What is NPM? What is the role of the node_modules folder?

▼ Deep Explanation

1. NPM

a. installs node_modules

- It has all libraries and dependencies used to develop UI react application.



▼ Answer

1. NPM

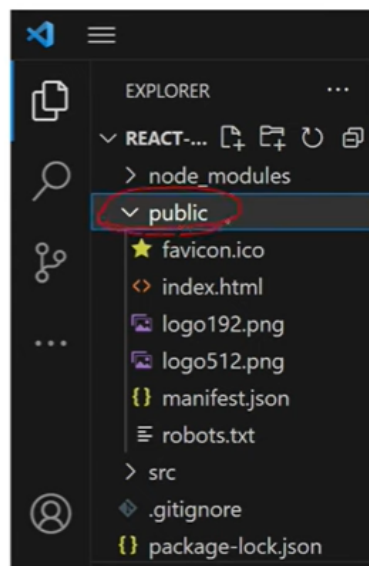
- ##### a. Node Package Manager is used to manage the dependencies for your React project, including the React library itself.

- b. node_modules folder contains all the dependencies of the project, including the React libraries.
- c. Loads node_modules folder.

▼ **Q2. What is the role of the public folder in React?**

▼ **Deep Explanation**

- 1. public folder
 - a. contains static files which don't get changed
 - favicon
 - images
 - index.html



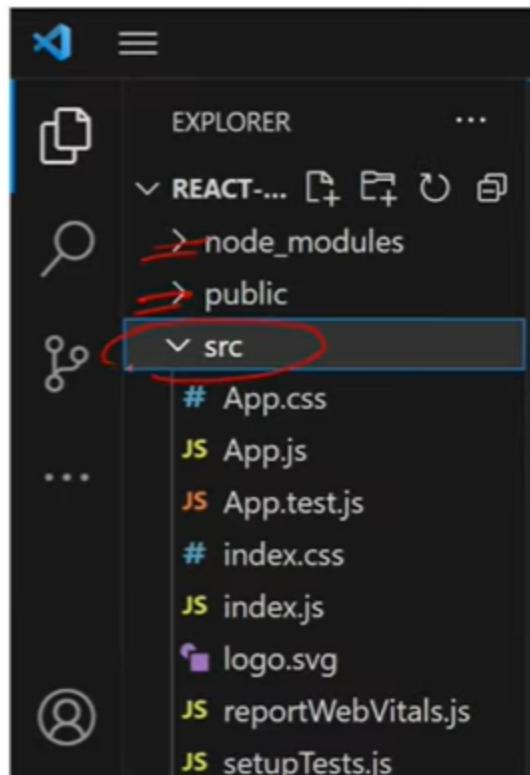
▼ **Answer**

- 1. Public folder contains static assets that are served directly to the user's browser, such as
 - a. images
 - b. fonts
 - c. index.html

▼ Q3. What is the role of the src folder in React?

▼ Answer

1. src folder
 - a. src folder is used to store all the source code of the application which is then responsible for the dynamic changes in your web application.



▼ Q4. What is the role of the index.html page in React?

▼ Answer

1. index.html
 - It is located inside the `public` folder because it is a static page.
 - When we say that React is a **single-page application**, this `index.html` is that main single page.
 - All the components in our React app are dynamically placed inside this page.

- When the user opens the browser, this is the first page that loads.
- The `div` with `id="root"` gets replaced by the components defined in the `App.js` file.
 - Here, the `div` with `id="root"` will be replaced by the component rendered in the `index.js` file.

The screenshot shows the VS Code Explorer on the left with the file tree expanded to `public > index.html`. The Editor on the right displays the content of `index.html`:

```

1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>React App</title>
5   </head>
6   <body>
7     <div id="root"></div>
8   </body>
9 </html>

```

▼ Q5. What is the role of the index.js file and ReactDOM in React?

▼ Deep Explanation

1. index.js

- When a React application runs, the React library starts by reloading the `index.js` file along with the `index.html` file.
- Inside the `index.js` file, we use the `ReactDOM` package

The screenshot shows the content of the `index.js` file:

```

JS index.js > ...
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

2. ReactDOM

- It contains a method called `createRoot()`.
- Using this method, we capture a reference to the root element (`div` with id `"root"`) from the `index.html` file and assign it to a variable:

```
const root = ReactDOM.createRoot(document.getElementById("root"))
```

- Then, we call the `render()` method on this variable (which holds the root reference):
 - Inside the `render()` method, we render the `App` component.
 - This results in the entire component structure being injected into the `index.html` file.
 - The `div` with `id="root"` is replaced with the `App` component and all of its child components, which are then displayed in the browser.

```
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
)
```

▼ Answer

1. ReactDOM

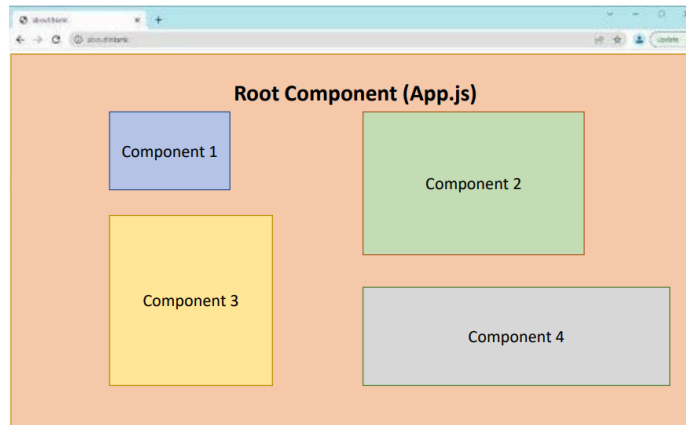
- a. ReactDOM is a JavaScript library that renders components to the DOM or browser.

2. index.js

- a. The index.js is the JavaScript file that replaces the root element of the index.html file with the newly rendered components.

▼ Q6. What is the role of the App.js file in React?

▼ Deep Explanation



1. Root Component

- a. It acts like a container for all child components.
- b. The `App.js` component:
 - Is the default root component in a React application.

2. Root App Component

- a. It is generated by default when we create a new React application.
- b. It is located inside the `src` (source) folder.
- c. We can add as many child components inside it as needed.

```
JS App.js > ...
import AppChild from "../Others/AppChild";

function App() {
  return (
    <div>
      <AppChild></AppChild>
    </div>
  );
}

export default App;
```

▼ Answer

1. App.js
 - a. Contain the root component(App) of react application.
2. App Component
 - a. Is like a container for other components.
3. App.js defines
 - a. the structure, layout, and routing in the application.

▼ Q7. What is the role of function and return inside App.js?

▼ Deep Explanation

1. Role of Function

- a. The `function` keyword is used to define a JavaScript function.
 - In React, a JavaScript function can represent a functional component.
- b. A function acts as a placeholder that contains all the logic of the component.
- c. The function takes `props` as arguments and returns **JSX** (JavaScript XML), which describes what should appear on the UI.

2. Role of Return

- a. `return` is used to return elements from a function.
- b. The UI we see in the browser comes from the `return` block.
- c. Typically, we return a `<div>` element and one or more child components.
- d. `index.js` along with the `ReactDOM` package renders these returned elements to the DOM, and the content becomes visible in the browser.

```
JS App.js > ...
import AppChild from "../Others/AppChild";

function App() {
  return (
    <div>
      <h1>Interview Happy</h1>
      <AppChild></AppChild>
    </div>
  );
}

export default App;
```

▼ Answer

1. The function keyword is used to define a JavaScript function that represents your React Component.
2. Function is like placeholder which contains all the code or logic of component.
3. The function takes in props as its argument and returns JSX.

▼ Q8. Can we have a function without a return inside App.js?

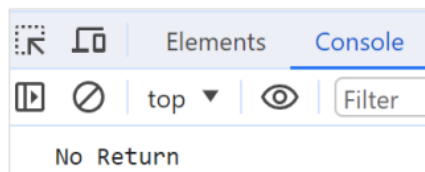
▼ Deep Explanation

1. If we don't use a `return` statement in a function, then nothing will be displayed in the browser.
2. However, we can still see output in the console without a `return` statement.


```
const FuncWithoutReturn = () => {
  //   return (
  //     <div>
  //       <h1>Interview Happy</h1>
  //     </div>
  //   );

  console.log("No Return");
};

export default FuncWithoutReturn;
```



▼ Answer

1. Yes, a function without a return statement is possible.
2. In that case, the component will not render anything in the UI.
3. A common use case is for logging purposes.
 - In real applications, we can have error logging components that do not return anything.

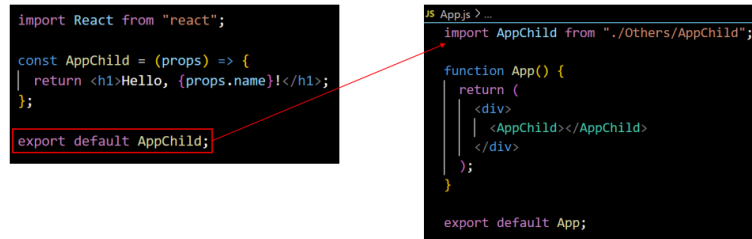
▼ Q9. What is the role of export default inside App.js?

▼ Deep Explanation

1. `export default` is placed at the end of the component file.
2. It is used to allow the component to be imported into other files.
3. If `export default` is removed, the component cannot be imported and it will cause an error.

▼ Answer

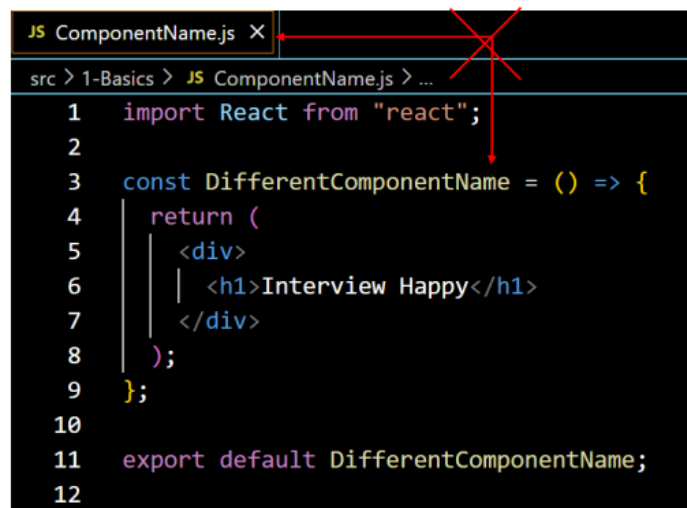
1. The `export` statement is used to make a component available for the `import` statement in other files.



▼ Q10. Do the file name and the component name have to be the same in React?

▼ Deep Explanation

1. No, the file name and component name can be different.
2. However, it's not recommended because it's better to keep the file name and component name the same.
 - a. This helps in maintaining easier, understandable, and organized code as per standard practices.



▼ Answer

1. No, the file name and component name don't have to be the same.
2. However, it is recommended to keep them same for easier to organize and understand your code.